

VICTORIA PARK ACADEMY SHENZHEN

CIE IGCSE COMPUTER SCIENCE (0478) — YEAR 9 SEMESTER 2

COURSE OVERVIEW & SCHEME OF WORK

Document Reference: VPA-CS-Y9S2-OVERVIEW-v1.0

Academic Year: 2025–2026

Target Examination: CIE IGCSE Computer Science (0478) — Paper 2 (Problem Solving & Programming)

Teaching Weeks: 18 weeks (plus review week)

Lesson Duration: 40 minutes

Prepared For: Class teachers, EAL coordinators, parents, students

SECTION 1 — COURSE SUMMARY & PHILOSOPHY

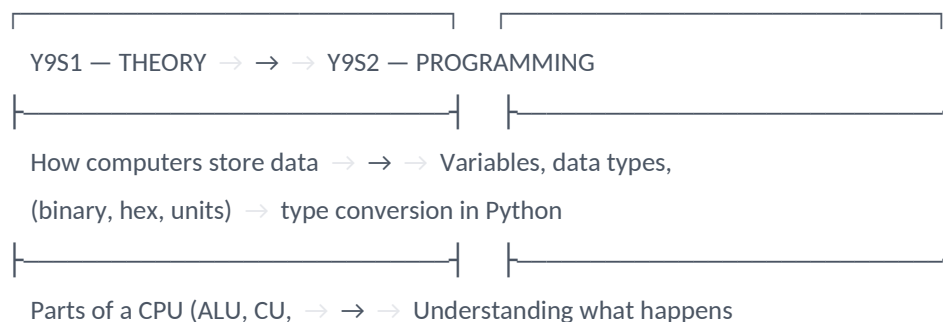
1.1 WHAT IS YEAR 9 SEMESTER 2?

Year 9 Semester 2 is the PROGRAMMING SEMESTER — the practical complement to the theory foundation built in Year 9 Semester 1 (Y9S1). While Y9S1 introduced the concepts of computer systems, data representation, and hardware, Y9S2 transforms those abstract ideas into working code. Every student will write, test, and debug real Python programs. By the end of this semester, every learner will be able to:

- Read and write pseudocode using CIE standard conventions
- Draw and interpret flowcharts using standard symbols
- Write structured Python programs using selection, iteration, and functions
- Manipulate lists (arrays), strings, and files confidently
- Trace code execution using dry-run tables
- Apply systematic testing strategies to their own programs
- Decompose problems using top-down design principles

1.2 CONNECTION TO YEAR 9 SEMESTER 1 (THEORY → PRACTICE)

Y9S1 laid the conceptual foundation. Y9S2 builds the practical superstructure.



registers, buses) → when code runs — registers

as variables, ALU as ops



Logic gates (AND, OR, NOT) → → → Boolean logic in selection

conditions and while loops



Operating systems → → → File handling — how OS

manages open/close/read/write



System security & ethics → → → Input validation,

defensive programming



This theory-to-practice bridge is made explicit in every unit. Teachers should regularly reference Y9S1 concepts to reinforce the spiral curriculum.

1.3 CONNECTION TO YEAR 10 & YEAR 11

Y9S2 directly prepares students for:

- Year 10 — Paper 2 (Problem Solving & Programming) intensive preparation
- Year 10 — Programming project (NEA-style skills, though 0478 is exam-only)
- Year 11 — Full mock examinations and final IGCSE examination

The Python skills practised this semester are EXACTLY the skills tested in

Paper 2 Questions 12–20. The pseudocode conventions taught are IDENTICAL to those used in CIE examination papers.

SECTION 2 — 18-WEEK CURRICULUM MAP



WEEK → TOPIC & LEARNING OBJECTIVES → LESSONS → ASSESS

1 → UNIT 0: SETTING UP & PYTHON RECAP → 2

- Install Thonny IDE; understand the REPL
- Recap print(), input(), basic variable use
- Introduction to commenting code (#)
- Setting up folder structure for CS work

中文: Python 环境设置, 复习基础语法

1–3 → UNIT 1: ALGORITHMS, PSEUDOCODE & FLOWCHARTS → 5 → QZ1

- Define algorithm; give examples from daily life → Wk3
- CIE pseudocode syntax: INPUT, OUTPUT, ←, IF, THEN, ELSE, ENDIF, REPEAT ... UNTIL, WHILE ...

DO ... ENDWHILE, FOR ... TO ... NEXT, PROCEDURE

- Flowchart symbols: terminal, process, decision, input/output, flow lines, start/end
- Decomposition: breaking problems into sub-tasks
- Trace tables: dry running simple algorithms
- Abstraction: identifying relevant information

中文: 算法, 伪代码, 流程图, 分解, 抽象

4-7 → UNIT 2: PYTHON PROGRAMMING — SELECTION & → 7 → QZ2
ITERATION → Wk7

- Variables: declaration, assignment, naming conventions (camelCase, snake_case)
- Data types: INTEGER, REAL, STRING, BOOLEAN, CHAR; type checking; type conversion
- Arithmetic operators: +, -, *, /, DIV, MOD, ^
- Comparison operators: =, <, >, <=, >=, <>
- Logical operators: AND, OR, NOT
- Selection: IF ... ELSE ... ENDIF, nested IF
- Iteration: FOR ... TO ... NEXT, REPEAT ... UNTIL, WHILE ... DO ... ENDWHILE
- Python equivalents: if/elif/else, for, while
- Common errors: infinite loops, off-by-one, indentation errors

中文: 选择结构, 循环结构, 运算符, 数据类型

8-10 → UNIT 3: DATA STRUCTURES — ARRAYS & STRINGS → 6 → QZ3

- 1D arrays (lists in Python): declaration, → Wk10
initialisation, accessing by index (0-based)
- Array operations: traversing, searching (linear search), inserting, deleting
- String handling: concatenation, length, substring extraction, upper/lower case
- ASCII character codes: ORD() and CHR()
- 2D arrays: declaration, accessing elements with two indices
- Practical: password validator, name formatter, simple database (list of records)

中文: 数组, 字符串, 列表, 二维数组, 搜索

11-13 → UNIT 4: FUNCTIONS & PROCEDURES → 6

- Built-in functions: print(), input(), len(),

range(), int(), float(), str(), ord(), chr()

- User-defined functions: def, return values
- Procedures vs functions (no return value)
- Parameters (formal) and arguments (actual)
- Local vs global scope; variable visibility
- Modular design: advantages of decomposition
- Parameter passing by value
- Library routines / modules: import, random, math (basic usage)

• Practical: menu system, maths quiz game, unit converter

中文: 函数, 参数, 返回值, 作用域, 局部变量, 全局变量

13-14 → UNIT 5: FILE HANDLING → 3 → QZ4

- Text file operations: OPEN, READ, WRITE, CLOSE → Wk14
- File modes: read ("r"), write ("w"), append ("a")

- Reading line by line; EOF loops
- CSV structure: reading/writing comma-separated data

- Error handling: file not found, permissions
- Practical: student marks processor, contact

list manager

中文: 文件处理, 读取, 写入, CSV 格式, 文件模式

15-16 → UNIT 6: TESTING, DEBUGGING & DOCUMENTATION → 5

- Types of error: syntax, logic, runtime
- Test data categories: normal, boundary (extreme), erroneous (abnormal / invalid)

- Trace tables: systematic dry-running
- Test plans: test number, description, input data, expected output, actual output, pass/fail

- Debugging strategies: print debugging, commenting-out, incremental testing
- Documentation: internal (comments), external (user guide, technical guide)

- The systems life cycle: analysis, design, coding, testing, maintenance, evaluation

中文: 测试, 调试, 错误类型, 测试数据, 文档编写

17-18 → UNIT 7: REVISION & END-OF-SEMESTER MOCK → 5 → MOCK

- Mixed-topic revision: all units 1–6 → Wk18
- Paper 2 examination technique: reading the question, planning the answer, time management
- Pseudocode-to-Python and Python-to-pseudocode translation practice
- Mock examination (1 hour, 50 marks)
- Post-mock feedback and target-setting for Y10

中文: 复习, 模拟考试, 考试技巧

TOTAL → 34 → 5

(+2 lessons for setup, +5 for revision/mock) → lessons → assess

SECTION 3 — STUDENT PROFILE & CONTEXT

3.1 COHORT CHARACTERISTICS

- Age range : 13–14 years (Year 9, Key Stage 3 transitioning to KS4)
- Language profile : Majority are EAL learners (English as Additional Language)
- First language : Predominantly Mandarin Chinese
- English level : B1–B2 on CEFR scale (intermediate to upper-intermediate)
- Prior CS exposure : Y9S1 Computer Science (theory), some may have done Scratch/basic Python in middle school
- Mixed ability : Significant spread; some students self-taught in Python, others completely new to text-based programming
- Class size : Typically 20–24 students
- Lesson length : 40 minutes (short — requires tight pacing)

3.2 IMPLICATIONS FOR TEACHING

- Every concept needs a VISUAL anchor (diagram, animation, physical demo)
- Code must be BROKEN INTO SMALL STEPS — no large blocks of code at once
- Bilingual keywords are NON-NEGOTIABLE — always display EN + 中文
- Pair programming is HIGHLY EFFECTIVE — EAL students process orally
- Physical code tracing (finger on paper) beats screen tracing for beginners
- Mini-whiteboards allow every student to respond simultaneously
- "Do Now" tasks should be RETRIEVAL-BASED, not new learning
- Homework must be CONSOLIDATION, not new content

SECTION 4 — ASSESSMENT CALENDAR



WEEK → ASSESSMENT → DURATION → FORMAT / FOCUS

Week 3 → Quiz 1 — Algorithms & → 15 min → 20 marks; MCQ,

Pseudocode → short answer,

problem solving

Week 7 → Quiz 2 — Python Programming → 20 min → 25 marks; MCQ,

(Selection & Iteration) → code reading,

code writing

Week 10 → Quiz 3 — Data Structures → 15 min → 20 marks; MCQ,

(Arrays & Strings) → code reading,

code writing

Week 14 → Quiz 4 — File Handling & → 15 min → 20 marks; MCQ,

Functions → code reading,

code writing

Week 18 → End-of-Semester Mock Exam → 60 min → 50 marks; Paper 2

(Paper 2 Style) → style; MCQ, code

tracing, problem

solving

ASSESSMENT WEIGHTING FOR REPORTING:

- Quiz 1 (Algorithms) : 10%
- Quiz 2 (Python Programming) : 15%
- Quiz 3 (Data Structures) : 10%
- Quiz 4 (File & Functions) : 10%
- End-of-Semester Mock Exam : 35%
- Classwork & Participation : 20%

SECTION 5 — STANDARD LESSON STRUCTURE (40 MINUTES)

Every lesson follows this RIGID structure to maximise learning in 40 minutes:



PHASE → TIME → ACTIVITY

DO NOW (Entry Task) → 3–5 min → Retrieval-based question on screen as

students enter. No new content.

Examples:

- "What does MOD do? Write one use."
- "Trace this loop: FOR i ← 1 TO 3"
- "Name the flowchart symbol for a decision."

Students write on mini-whiteboards;

teacher scans room for understanding.

DIRECT INSTRUCTION → 10–12 min → Teacher presents ONE new concept.

(I Do) → Uses:

- Visual diagram on board/projector
- Live coding (project Thonny)
- Bilingual keyword card on display
- Physical demonstration if possible

RULE: Show the CONCEPT first, then the CODE. Never show code without explaining WHY.

GUIDED PRACTICE → 10–12 min → Students work on a STRUCTURED task (We Do) → with teacher circulating.

Formats:

- "Fill in the missing line"
- "Predict the output" (discuss)
- Pair programming on one laptop
- Mini-whiteboard code sketching
- Trace table completion

Teacher identifies struggling students for immediate intervention.

INDEPENDENT PRACTICE → 10–12 min → Students work INDIVIDUALLY on: (You Do) → • Worksheet exercises

- Coding challenge (3 levels)
- Extension problem for early finishers

THREE LEVELS (students self-select or teacher assigns):

BRONZE — scaffolded, close support

SILVER — standard difficulty

GOLD — open-ended challenge

PLENARY (Exit Check) → 3–5 min → WHOLE-CLASS:

- "Show me on your whiteboard: What is the output of...?"
- "Tell your partner one thing you learned today."

- Thumbs up / sideways / down check

- Exit ticket (one question on slip)

HOMEWORK → (set in → ALWAYS consolidation. Never new lesson) → content. Typical tasks:

- Complete worksheet section
- Trace a program on paper
- Fix 3 bugs in given code

- Write 5 lines of pseudocode
- Vocabulary flashcard creation

SECTION 6 — EAL SUPPORT CHECKLIST

Every lesson MUST include the following EAL strategies:

☐ BILINGUAL KEYWORD DISPLAY

- Every new term introduced with: ENGLISH 中文 (pinyin)
- Keywords remain visible on wall/board for entire unit
- Colour-coded by unit (e.g., Unit 1 = blue, Unit 2 = green)

☐ VISUAL ANCHORS

- Diagrams for every abstract concept (variable as a box, array as a row)
- Flowchart symbols shown as physical shapes before symbols
- Icon set for common operations (loop arrow, decision diamond, etc.)

☐ SENTENCE STARTERS

- Printed on desk mats or displayed on board:
 - "The output will be _____ because _____"
 - "This loop runs _____ times"
 - "The variable X changes from _____ to _____"
 - "The error is _____ because _____"
 - "I need to use a _____ loop because _____"

☐ PAIR PROGRAMMING PROTOCOL

- Driver (types) and Navigator (watches/tells) swap every 5 minutes
- Both must be able to explain the code before moving on
- Mixed-ability pairing (strong EAL + weaker CS; strong CS + weaker EAL)

☐ MINI-WHITEBOARD ROUTINES

- Used at least TWICE per lesson (Do Now + Plenary minimum)
- All students show simultaneously — no hiding
- Teacher scans and gives immediate feedback

☐ CODE TRACING ON PAPER

- Printed code provided for EVERY trace-table exercise
- Students use coloured highlighters: pink = variables, yellow = output, green = conditions, blue = loops
- Physical finger-tracking encouraged before writing

☐ ORACY OPPORTUNITIES

- "Turn and tell your partner" built into every lesson
- Students must verbalise before writing code
- Chinese discussion permitted during pair work — concept first, then English

☐ DIFFERENTIATED LANGUAGE SUPPORT

- Word bank provided on every worksheet
- Bilingual instructions on all assessments

- Glossary at back of every booklet for reference

□ MODELLING & THINK-ALOUDS

- Teacher verbalises thinking while coding: "Now I need to check if the number is positive, so I need an IF statement..."
- Errors modelled positively: "Here's a mistake I made — let's fix it"

□ REPETITION & SPACED PRACTICE

- Key concepts revisited in Do Now tasks across multiple weeks
- Vocabulary tested in low-stakes quizzes every week

SECTION 7 — DIFFERENTIATION FRAMEWORK

All students are assessed against THREE attainment descriptors:

EMERGING (E)

- Can identify and describe basic programming concepts with support
- Can complete heavily scaffolded code (fill-in-the-blanks)
- Can trace simple algorithms with a provided trace table
- Beginning to use correct vocabulary (may mix English and Chinese)
- Requires teacher guidance to start tasks
- Produces partially complete programs with significant support

SUPPORT STRATEGIES:

- Sentence starters provided for all written responses
 - Code templates with clear gaps to fill
 - Pre-printed trace tables with column headers given
 - Partner work for all coding tasks
 - Additional visual aids and step-by-step guides
 - Extra time (25% additional) on all assessments
 - One-to-one check-in at start of independent practice
-

DEVELOPING (D)

- Can explain programming concepts in own words
- Can write simple programs independently from a given specification
- Can trace algorithms and identify most errors
- Uses technical vocabulary correctly most of the time
- Can convert between pseudocode and Python for basic structures

- Sometimes needs hints to get started on complex problems

SUPPORT STRATEGIES:

- Partial scaffolding (some lines provided, some blank)
- Checklists for completing tasks
- Hints available but not mandatory
- Occasional pair work for complex tasks
- Extension problems available for early finishers

SECURE (S)

- Can independently design, write, test, and debug programs
- Can convert fluently between pseudocode and Python
- Can create trace tables from scratch for given algorithms
- Uses technical vocabulary precisely and consistently
- Can decompose problems and select appropriate data structures
- Can explain code to others and evaluate alternative solutions
- Ready for Year 10 Paper 2 preparation

EXTENSION STRATEGIES:

- Open-ended problem-solving tasks
- Peer teaching opportunities
- Additional challenge: optimise solutions, use more efficient algorithms
- Introduction to concepts beyond the syllabus (e.g., recursion preview)
- Leadership in pair programming as the "expert" partner

ATTAINMENT TRACKING:

Each student's attainment is recorded on a grid for each of the 18 topics.

A student is deemed "Secure" for the semester if they achieve Secure on at least 12 of 18 topics AND pass the end-of-semester mock exam ($\geq 60\%$).

SECTION 8 — RESOURCE REQUIREMENTS

8.1 HARDWARE & SOFTWARE

REQUIRED ITEM	PURPOSE
• Thonny IDE (Python 3.10+) (free, cross-platform)	• Primary programming environment • Install on all student laptops
→ https://thonny.org	

- Student laptops (1 per student) Programming practice; research (Windows/Mac/Chromebook)
- Projector + teacher laptop Live coding demonstrations
- Mini-whiteboards (1 per student) Rapid assessment, code sketching, + markers planning before typing
- Coloured highlighters (3/pkg) Code tracing: pink=var, yellow=output, green=condition, blue=loop

8.2 PRINTED MATERIALS

REQUIRED ITEM	PURPOSE
---------------	---------

- | | |
|---|--|
| • Pseudocode templates (A4) | Standard CIE pseudocode reference cards
→ See Appendix A for template for student desks |
| • Flowchart symbol templates | Stencil/diagram for drawing flowcharts
→ See Appendix B in books and on whiteboards |
| • Printed code for tracing (provided in worksheets) | Every trace-table exercise needs code on paper — NEVER trace from screen |
| • Trace table blanks (with and without headers) | Pre-formatted tables with/without column headers for differentiation |
| • Bilingual keyword wall cards (A5, laminated) | Displayed by unit, colour-coded |
| • Sentence starter desk mats (bilingual) | Printed on A3, laminated for re-use |

8.3 DIGITAL RESOURCES

- Python starter files (.py) Pre-loaded with comments and skeleton code
- Worksheet masters (PDF/Word) For printing or digital distribution
- Quiz files (printed only) Security — no digital copies to students
- Mark schemes (teacher only) Secure storage

8.4 RECOMMENDED ONLINE RESOURCES

- CIE Syllabus (0478): <https://www.cambridgeinternational.org>
- Python documentation: <https://docs.python.org/3/>
- W3Schools Python: <https://www.w3schools.com/python/>
- BBC Bitesize Computer Science: <https://www.bbc.co.uk/bitesize/subjects/>
- CS Unplugged (algorithms): <https://www.csunplugged.org>

SECTION 9 — KEY BILINGUAL GLOSSARY (50+ TERMS)

This glossary contains ALL programming terms taught in Y9S2. Each term is presented with: ENGLISH | 中文 | pinyin | brief definition.

Display these as wall cards by unit. Students should have a copy in their

booklets.

UNIT 1 — ALGORITHMS & PROBLEM SOLVING

algorithm	算法	suàn fǎ	A step-by-step set of instructions to solve a problem
pseudocode	伪代码	wěi dài mǎ	A structured way of writing an algorithm using English-like notation
flowchart	流程图	liú chéng tú	A diagram using symbols to show the steps and decisions in an algorithm
decomposition	分解	fēn jiě	Breaking a complex problem into smaller, manageable sub-problems
abstraction	抽象	chōu xiàng	Removing unnecessary detail to focus on what is important
algorithmic thinking	算法思维	suàn fǎ sī wéi	The ability to think in terms of step-by-step solutions
stepwise refinement	逐步细化	zhú bù xì huà	Developing an algorithm by gradually adding more detail
dry run	手工运行/桌面检查	shǒu gōng yùn xíng	Manually tracing through an algorithm to check it works
trace table	跟踪表	gēn zōng biǎo	A table used to record the values of variables during a dry run
terminal symbol	起止符	qǐ zhǐ fú	The oval shape in a flowchart that marks the start and end
process symbol	处理符	chǔ lǐ fú	The rectangle in a flowchart that represents an action or calculation
decision symbol	判断符	pàn duàn fú	The diamond shape in a flowchart that represents a yes/no decision
input/output symbol	输入/输出符	shū rù/shū chū fú	The parallelogram in a flowchart for input or output operations
flow line	流程线	liú chéng xiàn	The arrows in a flowchart that show the direction of flow
subroutine	子程序	zǐ chéng xù	A named block of code that can be called from multiple places

UNIT 2 — PYTHON BASICS & CONTROL STRUCTURES

variable	变量	biàn liàng	A named storage location that holds a value which can change
----------	----	------------	--

constant	常量	cháng liàng	A named value that does not change during program execution
data type	数据类型	shù jù lèi xíng	The kind of data a variable can hold (e.g., integer, string)
integer	整数	zhěng shù	A whole number without decimal places (e.g., 42, -7)
real / float	实数/浮点数	shí shù / fú diǎn shù	A number with decimal places (e.g., 3.14, -0.5)
string	字符串	zì fú chuàn	A sequence of characters (text) enclosed in quotes
Boolean	布尔值	bù ěr zhí	A data type with only two possible values: TRUE or FALSE
character / char	字符	zì fú	A single letter, digit, or symbol
input	输入	shū rù	Data entered into a program by the user
output	输出	shū chū	Data produced by a program and displayed to the user
assignment	赋值	fù zhí	Giving a value to a variable using ← (pseudocode) or = (Python)
comment	注释	zhù shì	Explanatory text in code ignored by the computer (# in Python)
arithmetic operator	算术运算符	suàn shù yùn suàn fú	Mathematical symbols: +, -, *, /, DIV, MOD, ^
comparison operator	比较运算符	bǐ jiào yùn suàn fú	Symbols that compare values: =, <, >, <=, >=, <>
logical operator	逻辑运算符	luó jí yùn suàn fú	Boolean operators: AND, OR, NOT
AND	与/并且	yǔ / bìng qiě	Logical operator: TRUE only if BOTH conditions are TRUE
OR	或/或者	huò / huò zhě	Logical operator: TRUE if AT LEAST ONE condition is TRUE
NOT	非/不	fēi / bù	Logical operator: reverses TRUE to FALSE and vice versa
MOD	取模/求余	qǔ mó / qiú yú	Operator that gives the remainder after division (e.g., 17 MOD 5 = 2)
DIV	整除	zhěng chú	Operator for integer division (e.g., 17 DIV 5 = 3)
selection	选择结构	xuǎn zé jié gòu	A control structure that chooses between different paths (IF statements)

if	如果	rú guǒ	Keyword that starts a conditional statement
else	否则	fǒu zé	Keyword that defines what happens if the condition is FALSE
elif / else if	否则如果	fǒu zé rú guǒ	Keyword for checking a second condition if the first is FALSE
condition	条件	tiáo jiàn	A Boolean expression that determines which path to take (e.g., $x > 5$)
nested	嵌套的	qiàn tà de	Placed inside another structure (e.g., an IF inside an IF)
iteration	迭代/循环	dié dài / xún huán	Repeating a block of code multiple times
for loop	for 循环	for xún huán	A count-controlled loop that repeats a set number of times
while loop	while 循环	while xún huán	A condition-controlled loop that repeats while a condition is TRUE
range	范围	fàn wéi	A function that generates a sequence of numbers (in Python)
infinite loop	无限循环	wú xiàn xún huán	A loop that never stops because the condition never becomes FALSE
nested loop	嵌套循环	qiàn tà xún huán	A loop inside another loop
counter-controlled	计数控制	jì shù kòng zhì	A loop that repeats a predetermined number of times
condition-controlled	条件控制	tiáo jiàn kòng zhì	A loop that repeats based on a condition being TRUE or FALSE
post-condition loop	后测试循环	hòu cè shì xún huán	A loop that checks the condition AFTER each iteration (REPEAT UNTIL)
pre-condition loop	前测试循环	qián cè shì xún huán	A loop that checks the condition BEFORE each iteration (WHILE)
indentation	缩进	suō jìn	The spaces at the start of a line that show code structure in Python
code block	代码块	dài mǎ kuài	A group of statements that belong together (indented in Python)
casting / type	类型转换	lèi xíng zhuǎn huàn	Converting a value from one data type to another (e.g., <code>int(x)</code>)
conversion			

UNIT 3 — DATA STRUCTURES: ARRAYS & STRINGS

array	数组	shù zǔ	A collection of data items of the same type stored under one name
list (Python)	列表	liè biǎo	Python's implementation of an array (can hold mixed types)
index	索引/下标	suǒ yǐn / xià biāo	The position number of an element in an array (starts at 0 in Python)
element	元素	yuán sù	A single item stored in an array
2D array	二维数组	èr wéi shù zǔ	An array with rows and columns, like a table or grid
append	添加/追加	tiān jiā / zhuī jiā	To add an element to the end of a list
remove	删除	shān chú	To delete an element from a list
insert	插入	chā rù	To add an element at a specific position in a list
length	长度	cháng dù	The number of elements in an array or characters in a string
concatenation	连接/拼接	lián jiē / pīn jiē	Joining two strings together end-to-end (e.g., "Hello" + "World")
slicing	切片	qiē piàn	Extracting a portion of a string or list using index ranges
substring	子字符串	zǐ zì fú chuàn	A continuous portion extracted from a larger string
ASCII	ASCII 码	ASCII mǎ	American Standard Code for Information Interchange — numeric codes for characters
ORD()	取 ASCII 码	qǔ ASCII mǎ	Function that returns the ASCII code of a character
CHR()	取字符	qǔ zì fú	Function that returns the character for a given ASCII code
searching	搜索	sōu suǒ	Looking through an array to find a specific value
linear search	线性搜索	xiàn xìng sōu suǒ	Checking each element in order until the target is found
traversing	遍历	biàn lì	Going through every element in an array one by one

UNIT 4 — FUNCTIONS & PROCEDURES

function	函数	hán shù	A named block of code that performs a task and returns a value
procedure	过程/子程序	guò chéng / zǐ chéng xù	A named block of code that performs a task but does NOT return a value
parameter	形参/参数	xíng cān / cān shù	A variable declared in the function definition that receives a value
argument	实参/参数值	shí cān / cān shù zhí	The actual value passed to a function when it is called
return value	返回值	fǎn huí zhí	The value sent back by a function to the caller
scope	作用域	zuò yòng yù	The region of a program where a variable can be accessed
local variable	局部变量	jú bù biàn liàng	A variable declared inside a function, only accessible there
global variable	全局变量	quán jú biàn liàng	A variable declared outside all functions, accessible everywhere
module	模块	mó kuài	A file containing Python code (functions, variables) that can be imported
library / library	库/函数库	kù / hán shù kù	A collection of pre-written functions and modules for reuse
routine			
import	导入	dǎo rù	To bring in code from another module for use in your program
built-in function	内置函数	nèi zhì hán shù	A function that is part of the programming language (e.g., print(), len())
user-defined function	用户定义函数	yòng hù dìng yì hán shù	A function written by the programmer
call / invoke	调用	diào yòng	To execute a function by using its name with parentheses
pass by value	值传递	zhí chuán dì	Passing a copy of a value to a function (original is not changed)

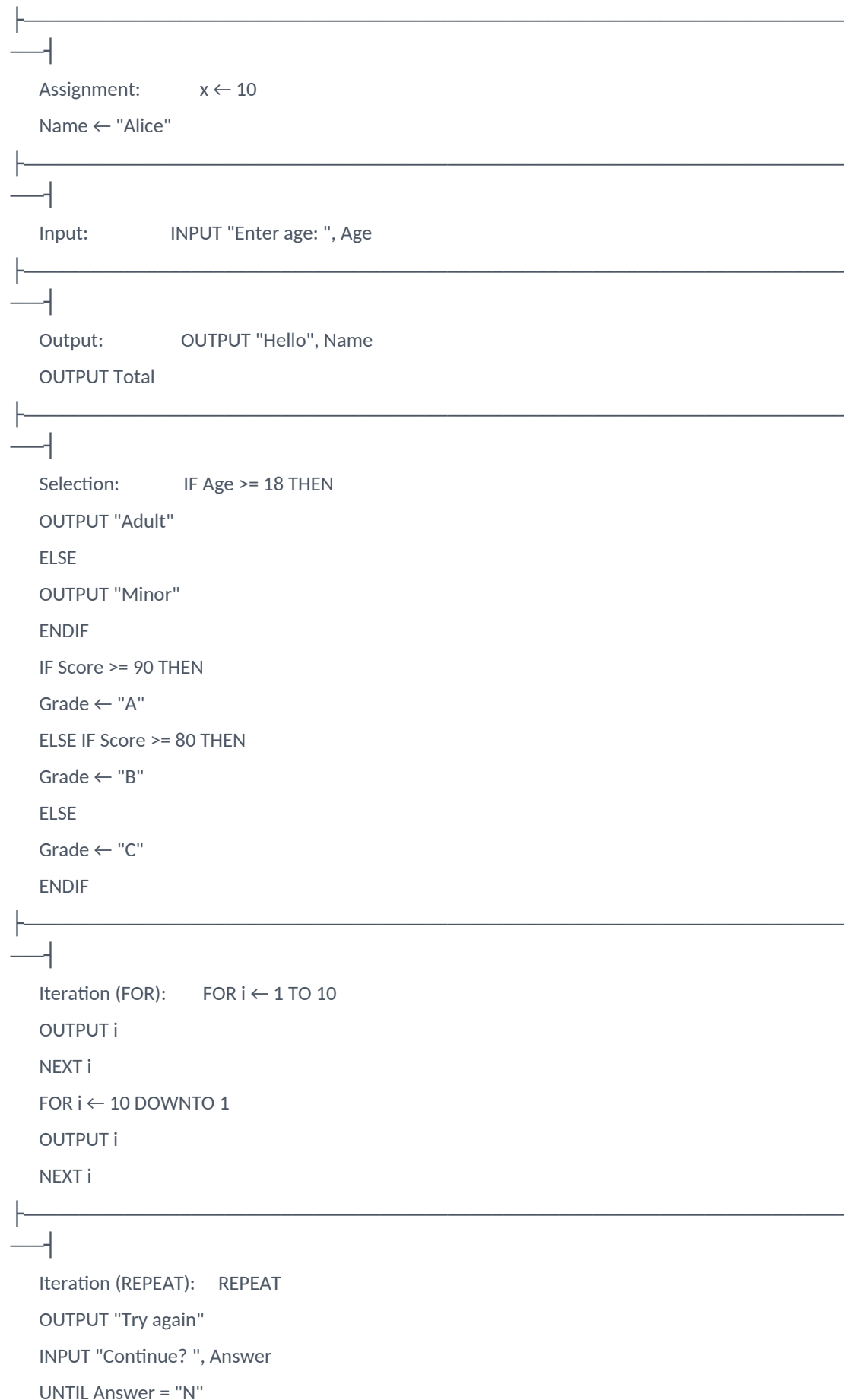
UNIT 5 — FILE HANDLING

file	文件	wén jiàn	A collection of data stored on a computer with a name
read	读取	dú qǔ	To retrieve data from a file

write	写入	xiě rù	To save data to a file (overwrites existing content)
append	追加	zhuī jiā	To add data to the end of an existing file without deleting
open	打开	dǎ kāi	To make a file available for reading or writing
close	关闭	guān bì	To finish using a file and release it back to the operating system
CSV	CSV 格式	CSV gé shì	Comma-Separated Values — a simple format for storing tabular data
file mode	文件模式	wén jiàn mó shì	Specifies how a file is opened: "r" (read), "w" (write), "a" (append)
EOF	文件结束	wén jiàn jié shù	End Of File — the condition when all data in a file has been read
record	记录	jì lù	A set of related fields in a file (one row in a table)
field	字段	zì duàn	A single data item within a record (one column in a table)
delimiter	分隔符	fēn gé fú	A character that separates fields in a file (usually a comma in CSV)

UNIT 6 — TESTING, DEBUGGING & DOCUMENTATION

test plan	测试计划	cè shì jì huà	A document describing what will be tested, how, and with what data
dry run	手工运行	shǒu gōng yùn xíng	Manually working through an algorithm to check for errors
syntax error	语法错误	yǔ fǎ cuò wù	A mistake in the grammar/rules of the programming language
logic error	逻辑错误	luó jí cuò wù	A mistake where the program runs but produces wrong results
runtime error	运行时错误	yùn xíng shí cuò wù	An error that occurs while the program is running (e.g., division by zero)
normal data	正常数据	zhèng cháng shù jù	Test data that is typical, valid, and expected
boundary data	边界数据	biān jiè shù jù	Test data at the limits of acceptable values (also called extreme data)
erroneous data	错误数据	cuò wù shù jù	Test data that is invalid to check error handling (also called abnormal)



Iteration (WHILE): WHILE Count < 5 DO
OUTPUT Count
Count $\leftarrow$ Count + 1
ENDWHILE

Arrays: DECLARE Names : ARRAY[1:10] OF STRING
Names[1] $\leftarrow$ "Alice"
FOR i $\leftarrow$ 1 TO 10
OUTPUT Names[i]
NEXT i

Functions/Procedures: PROCEDURE Greet(Name)
OUTPUT "Hello", Name
ENDPROCEDURE
FUNCTION Add(a, b)
RETURN a + b
ENDFUNCTION
CALL Greet("Bob")
Result $\leftarrow$ Add(3, 4)

File Handling: OPENFILE "data.txt" FOR READ
READFILE "data.txt", Line
WRITEFILE "output.txt", "Hello"
CLOSEFILE "data.txt"

Built-in Functions: LENGTH("Hello") $\rightarrow$ 5
SUBSTRING("Hello", 2, 3) $\rightarrow$ "ell"
UCASE("hi") $\rightarrow$ "HI"
LCASE("HI") $\rightarrow$ "hi"
ROUND(3.7) $\rightarrow$ 4
RANDOM() $\rightarrow$ 0 to 1 (random decimal)
INT(x) $\rightarrow$ integer part of x
MOD(x, y) $\rightarrow$ remainder of $x \div y$

DIV(x, y) → integer division $x \div y$

ORD("A") → 65

CHR(65) → "A"

APPENDIX B — FLOWCHART SYMBOL REFERENCE

FLOWCHART SYMBOLS

 Start / End (Terminal)

Oval shape. Every flowchart has ONE start and ONE end.

 Input / Output (Parallelogram)

/ / Any operation that inputs or outputs data.

 Process (Rectangle)

Any calculation or assignment.

Example: Total ← Price * Quantity

 Decision (Diamond)

/ \ A yes/no or true/false question.

/ \ Has TWO outgoing arrows: YES/NO or TRUE/FALSE

▼ ▼ Example: Is Score > 50?

 Flow Line (Arrow)

Shows direction of flow. Always use arrows!

 Pre-defined Process (Rectangle with double sides)

|| || A call to a sub-procedure or function.

|| || Example: CALL CalculateTotal

○ Connector (Circle)

Used to connect flowcharts across pages.

APPENDIX C — DIFFERENTIATED TASK EXAMPLE (ONE LESSON)

LESSON CONTEXT: Writing a program to find the largest of three numbers

BRONZE — EMERGING

Complete this program by filling in the gaps:

```
num1 = int(input("Enter first number: "))
```

```
num2 = int(input("Enter second number: "))
```

```

num3 = int(input("Enter third number: "))
largest = _____ # Hint: start by assuming num1 is largest
if num2 > largest:
    largest = _____
if _____ > largest:
    largest = _____
print("The largest number is", _____)

```

SILVER — DEVELOPING

Write a program that asks the user for three numbers and prints the largest.

Use IF and ELSE. Add validation: reject numbers less than 0.

HINTS:

1. Get three inputs and convert to integers
2. Use nested IF statements to compare
3. Use an IF to check if any number is negative

GOLD — SECURE

Write a program that finds the largest of ANY number of values (the user first enters how many numbers they want to compare).

CHALLENGES:

1. Use a list to store all numbers
2. Use a loop to find the maximum without using Python's max() function
3. Add input validation (reject non-numeric and negative inputs)
4. Display the result with a formatted message
5. Ask the user if they want to try again (REPEAT...UNTIL structure)

APPENDIX D — CODE TRACING COLOUR-KEY

Students should use this colour system when tracing code on paper:

COLOUR → USE FOR

PINK → Variables — underline each variable change

and write its new value in the trace table

YELLOW → Output — highlight every OUTPUT or print() statement

GREEN → Conditions — circle every IF/WHILE condition and evaluate it (TRUE/FALSE)

BLUE → Loops — draw a bracket around each complete loop iteration and number it

ORANGE → Function calls — underline function calls and draw arrows to where they go

APPENDIX E — SCHEME OF WORK SUMMARY TABLE

Week	Unit	Key Learning	Assessment
1	0. Setup + U1	Thonny, pseudocode basics	
2	U1. Algorithms	Flowcharts, decomposition	
3	U1. Algorithms	Trace tables, dry runs	Quiz 1 (20 marks)
4	U2. Python Basics	Variables, data types, I/O	
5	U2. Python Basics	Operators, IF statements	
6	U2. Python Basics	FOR loops, WHILE loops	
7	U2. Python Basics	Nested structures, errors	Quiz 2 (25 marks)
8	U3. Arrays	1D arrays, lists, indexing	
9	U3. Arrays/Strings	String handling, ASCII	
10	U3. Arrays/Strings	2D arrays, linear search	Quiz 3 (20 marks)
11	U4. Functions	Built-in, user-defined	
12	U4. Functions	Scope, parameters, return	
13	U5. File Handling	Open, read, write, close	
14	U5. File Handling	CSV, error handling	Quiz 4 (20 marks)
15	U6. Testing	Error types, test plans	
16	U6. Testing	Documentation, life cycle	
17	U7. Revision	Mixed topic practice	
18	U7. Mock Exam	Paper 2 mock examination	Mock (50 marks)

END OF DOCUMENT

Document: VPA-CS-Y9S2-OVERVIEW-v1.0

Prepared: Victoria Park Academy Shenzhen — Computer Science Department

Syllabus: CIE IGCSE Computer Science (0478)

For Use: Academic Year 2025–2026, Year 9 Semester 2